

Interface Interview Questions and Answers in C#

In this article, I am going to discuss **Interface Interview Questions and Answers in C#** with Examples. Please read our previous article, where we discussed [Interface in C#](#). You will get lots of questions in the interview based on the Interface Concepts. So, in this article, I am going to discuss the most frequently asked Interface Interview Questions and Answers in C#.

1. **What is an Interface in C#?**
2. **What are the different types of Inheritance Supported by C#?**
3. **Why do we need an interface in C#?**
4. **Can I use public access specifiers for interface methods in C#?**
5. **Can an Interface Implement an Abstract Class in C#?**
6. **Can an Interface be Declared as Sealed in C#?**
7. **Is more than one Interface allowed to Implement a Class in C#?**
8. **Is it Necessary to Implement all Interface Methods in C#?**
9. **How Interface is Different from a Class in C#?**
10. **What are the Similarities Between the Interface and Abstract Class in C#?**
11. **What is the Difference Between Interface and Abstract Class in C#?**
12. **What are the Advantages of using Interface in C#?**

What is an Interface in C#?

The Interface in C# is a **fully un-implemented class** used for declaring a set of methods of an object. So, we can define an interface as a **pure abstract class** which allows us to define only abstract methods. The abstract method means a method without a body or implementation.

```
interface IInterfaceName
{
    void Method1();
    void Method2();
}
```

It is used to achieve **multiple inheritances** which can't be achieved by class. It is used to achieve full abstraction because it cannot have a method body.

```

interface IInterface1
{
    void Method1();
    void Method2();
}
interface IInterface2
{
    void Method1();
    void Method2();
}
//Multiple Inheritance
class Class1 : IInterface1, IInterface2
{
    public void Method1() { }
    public void Method2() { }
}

```

Its implementation must be provided by class or struct. The class or struct which implements the interface must provide the implementation of all the methods declared inside the interface.

What are the different types of Inheritance Supported by C#?

A class can be inherited either from another class or from an interface also. So, inheritance can be divided into two categories

1. **Implementation Inheritance**
2. **Interface Inheritance**

If a class is inheriting from another class, we call it implementation inheritance and the main concept of implementation inheritance is child classes can consume the members of their parent class.

```

class Class1
{
    public void Method1() { }
}
//Implementation Inheritance
class Class2 : Class1
{
    public void Method2() { }
}

```

On the other hand, if a class is inheriting from an interface, we call it Interface Inheritance but interface inheritance does not provide any reusability because here we are not consuming the members of the parent under the child. The child is only implementing the parent's members.

```

interface IInterface1
{
    void Method1();
    void Method2();
}
//Interface Inheritance
class Class2 : IInterface1
{
    public void Method1() { }
    public void Method2() { }
}

```

Why do we need an interface in C#?

We know the concept of multiple inheritances where one class is derived from more than one superclass. For example, a definition like

```

class B
{
}
class C
{
}
class A: B, C
{
}

```

But this concept is not supported by .NET with classes. Since a large no of real-time applications require the use of multiple inheritances, where we inherit properties and behaviors from several different classes. That's why .NET provides an alternative approach known as the interface to support the concept of multiple inheritances.

Can I use public access specifiers for interface methods in C#?

.NET interface methods are implicitly public by default, even if they belong to nested interfaces. Non-public modifiers are not valid for interface methods. So, the compiler will fail and warn you in this case. Nested interfaces may be declared protected or private but not the interface methods. So, if you try to declare the method with public access specifier, you will get the following error.

```
interface ITestInterface1
{
    public void Method1();
}
```

 void ITestInterface1.Method1()

The modifier 'public' is not valid for this item

Can an Interface Implement an Abstract Class in C#?

No. In .NET an interface cannot implement an abstract class. An interface may only extend a super interface. However, an abstract class can implement an interface because an abstract class can contain both abstract methods and concrete methods. If you try to implement an interface you will get the following compile-time error.

```
public abstract class A
{
    public abstract void Method1();
}
interface ITestInterface1 : A
{
}
```

 class AbstractClassMethods.A

Type 'A' in interface list is not an interface

Can an Interface be Declared as Sealed in C#?

No, it is not permitted to declare an interface as sealed; it will cause a compilation error. This is a .NET language design decision. Interface types are intended to be implemented and can be extended without restriction. If you try to declare the interface as sealed, you will get the following error.

```
sealed interface ITestInterface1
{
}
```

 interface AbstractClassMethods.ITestInterface1

The modifier 'sealed' is not valid for this item

[Show potential fixes](#) (Alt+Enter or Ctrl+.)

Is more than one Interface allowed to Implement a Class in C#?

Yes, a class can implement multiple interfaces; this is an effective way to achieve multiple inheritances in C#. But a class can extend only one superclass. For a better understanding, please have a look at the following example.

```
interface ITestInterface1
{
    void Method1();
}
interface ITestInterface2
{
    void Method2();
}

//Class A Implementing two interfaces
public class A : ITestInterface1, ITestInterface2
{
    public void Method1()
    {
        //Method1 Implementation
    }

    public void Method2()
    {
        //Method2 Implementation
    }
}
```

Is it Necessary to Implement all Interface Methods in C#?

It is not necessary for a class that implements an interface to implement all its methods, but in this case, the class must be declared as abstract. For a better understanding, please have a look at the following code.

```
//Class A Implementing one method and hence
//declared as abstract
public abstract class A : ITestInterface1
{
    //Implement Method1
    public void Method1()
    {
        //Method1 Implementation
    }

    //Declare Method2 as abstract
    public abstract void Method2();
}
```

How Interface is Different from a Class in C#?

An interface is different from a class in the following ways:

1. We cannot instantiate an interface.
2. An interface does not contain any constructor or data fields or destructor, etc.
3. All of the methods of an interface are abstract and public by default.
4. An interface is not extended by a class; it is implemented by a class.
5. An interface can extend multiple interfaces.

What are the Similarities Between the Interface and Abstract Class in C#?

An interface is similar to an abstract class in the following ways

1. Both interface and the abstract class cannot be instantiated means we cannot create the object.
2. But we can create a reference variable for both the interface and abstract class.
3. The subclass should implement all abstract methods.
4. Both cannot be declared sealed.

What is the Difference Between Interface and Abstract Class in C#?

The main difference to be answered in the interview is as follows. The interface is a **fully un-implemented** class used for declaring a set of methods of an object. The abstract class is a **partially implemented** class. It implements some of the methods of an object. These implemented methods are common for all next-level subclasses. The remaining operations are implemented by the next-level subclasses according to their requirement.

The interface allows us to develop **multiple inheritances**. So, we must start object design with interface whereas abstract class does not support multiple inheritances so it always comes next to interface in the object creation process.

Abstract Class:

1. It is a partially implemented class. It allows us to define both concrete and abstract methods.
2. It should be declared as abstract by using the abstract keyword, abstract methods should also contain the abstract keyword.
3. Its member's default accessibility modifier is private and can be changed to any of the other accessibility modifiers.
4. It is possible to declare data fields in an abstract class.
5. An abstract class can contain a non-abstract function.
6. An abstract class can inherit from another abstract class or from an interface.
7. An abstract class cannot be used to implement multiple inheritances.
8. Abstract class members can have Access Specifiers.

Interface:

1. It is a fully un-implemented class. It allows us to define only abstract methods.
2. It should be created by using the keyword interface. By default, all the members are abstract only. Explicitly using abstract keyword is not allowed.
3. Its member's default accessibility modifier is public and cannot be changed.
4. It is not possible to declare any data fields in an interface.
5. An interface cannot contain non-abstract functions.
6. An interface can inherit from only other interfaces but cannot inherits from the abstract class.
7. An interface can be used to implement multiple inheritances.
8. Interface members cannot have Access Specifiers.

What are the Advantages of using Interface in C#?

The following are the advantages of using Interface in the C# application.

1. It is used to achieve loose coupling.
2. It is used to achieve total abstraction.
3. To achieve multiple inheritance and abstraction.

In the next article, I am going to discuss [Interface Realtime Examples in C#](#). Here, in this article, I try to explain **Interface Interview Questions and Answers in C#** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Interface Interview Questions and Answers in C# with Examples article.